16.08.26

P. Sai Akash
192521353

CSA0311 DATA STRUCTURES

CO4 - AT1 - COMPARATIVE ANALYSIS

1. Compare binary heaps and Fibonacci heaps for implementing Dijkstra's shortest path algorithm on a dense graph with 100,000 nodes and 10 million edges. Analyze amortized time complexity for decrease key operations, memory overhead, and practical performance. Why is Fibonacci heap rarely used in practice despite better theoretical bounds?

Consider a dense graph with,

* Vertices (V) = 100,000
* Edges (E) = 10,000,000

Feature	Binary heap	Fibonacci Heap
Insert	$O(\log V)$	$O(1)$
Extract-Min	$O(\log V)$	$O(\log V)$
Decrease-key	$O(\log V)$	$O(1)$
Dijkstra complexity	$O((V+E)\log V)$	$O(E + V\log V)$
Memory usage	low	High
Implementation	Simple	Very complex
Practical performance	Usually better	Often slower

Binary Heap.

For Dijkstra's algorithm, the complexity is,

$O((V+E) \log V)$

Since the graph has 10 million edges, many decrease-key operations are required,

$O(10,000,000 \times (\log 100,000))$

Fibonacci Heap.

$O(E + V \log V)$

The main advantage is that Decrease-key takes O(1) amortized time.

Memory Overhead,

Binary Heap

* Usually implemented using an array
* Each node requires very little additional memory.

Fibonacci Heap

* Uses multiple pointers for parent, child, sibling, and other relationship.
* Requires more memory per node.

2. Compare Timsort and traditional merge sort for sorting real-world data with following characteristics, (a) already sorted array. (b) reverse sorted array. (c) Array with many duplicate keys. (d) array with random order.

Timsort is a hybrid sorting algorithm that combines Merge sort and Insertion sort. It detects already ordered portions of the input, called runs, making it adaptive.

Algorithm	Best Case	Average Case	Worst case
Tim Sort	$O(n)$	$O(n \log n)$	$O(n \log n)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

(a) Already sorted array.

* Timsort detects that the entire array is already sorted. $O(n)$.
* Merge sort still divides the array and merge all subarrays. $O(n \log n)$

(b) Reverse Sorted Array.

* Timsort can detect descending runs and reverse them. Therefore, it can take advantage of the existing reverse order. $O(n)$
* Merge sort performs the same divide and merge process. $O(n \log n)$

(c) Array with many Duplicate keys.

* Timsort efficiently merges existing runs and is stable. Duplicate elements are handled efficiently $O(n \log n)$